Lesson 10 Teacher's Key: Play It Again

TIC-80 - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

The simplest possible version of 'factor setup into a reusable function' this week - a plain Lua function, called once at cart-load time and again from inside TIC() on restart, with no additional engine machinery needed at all.

Level 1 - Full Worked Solution

- start_game() is the old top-level setup code, unchanged in content, just wrapped in a function definition and called explicitly (once outside TIC(), and again inside it on restart).
- keyp(18) - not key(18) - is essential here: keyp() only returns true on the exact frame R was first pressed, so calling start_game() from it fires exactly once per press, not continuously while held.
- The 'if game_active then ... end' wrapper inside the enemy-drawing loop is the simplest possible freeze mechanism available in this engine: a single conditional around existing code, no special API needed.

Highlighted lines are new this week; everything else is code you already have.

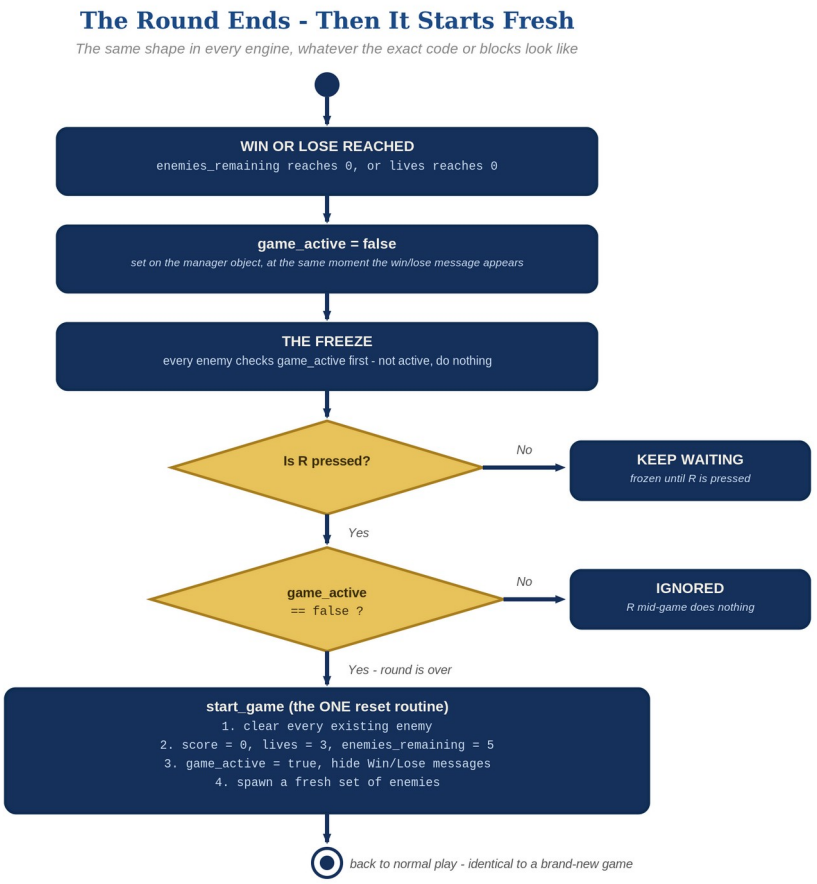
```
-- player position x and y are set above this, exactly as in earlier
lessons
game_active=true

function start_game()
 enemies={}
 for enemy_count=0,4 do
  table.insert(enemies,{ex=20+enemy_count*20,ey=10,alive=true})
 end
 bullets={}
 score=0
 lives=3
 enemies_remaining=5
 game_active=true
end

start_game()

function TIC()
 cls(0)
 if btn(3) then x=x+1 end
 if btnp(4) then
```

```
  table.insert(bullets,{bx=x,by=y})
 end
 for bullet_index=#bullets,1,-1 do
  local b=bullets[bullet_index]
  b.by=b.by-2
  local hit=false
  for enemy_index=1,#enemies do
   local e=enemies[enemy_index]
   if e.alive and b.bx<e.ex+8 and b.bx+8>e.ex and b.by<e.ey+8 and
b.by+8>e.ey then
    e.alive=false
    enemies_remaining=enemies_remaining-1
    score=score+10
    if enemies_remaining<=0 then game_active=false end
    hit=true
   end
  end
  if hit then
   table.remove(bullets,bullet_index)
  elseif b.by<0 then
   table.remove(bullets,bullet_index)
  else
   spr(2,b.bx,b.by,0)
  end
 end
 for enemy_index=1,#enemies do
  local enemy=enemies[enemy_index]
  if enemy.alive then
   if game_active then
    enemy.ey=enemy.ey+0.3
    if enemy.ey>136 then
     enemy.alive=false
     lives=lives-1
    if lives<=0 then game_active=false end
    end
   end
    spr(3,enemy.ex,enemy.ey,0)
  end
 end
 spr(1,x,y,0)
 print("Score: "..score,5,5,12)
 print("Lives: "..lives,5,15,12)
 if enemies_remaining<=0 then
  print("You Win! Press R to Restart",50,60,12)
 end
 if lives<=0 then
  print("Game Over - Press R to Restart",40,60,8)
 end
if keyp(18) and not game_active then
 start_game()
end
end
```



When the round ends, everything freezes - and R, pressed only while the round is over, runs the one reset routine. The same shape in every engine.

Why This Structure

- TIC-80 has no built-in concept of 'the game restarting' - this lesson builds the entire idea from three plain Lua ingredients (a function, a boolean flag, and an edge-triggered key check), which is a good, honest illustration of how much can be built from very few primitives.
- Verifying keyp()'s exact behavior (vs. key()) against the official API docs mattered specifically because this engine has had real, documented API-detail mistakes earlier in this course - the same discipline applied there was applied here.

Level 2 / Level 3 Talking Points

- Ask students to predict what would happen if key(18) were used instead of keyp(18) - start_game() would be called on every single frame R is held down, likely dozens of times per second, resetting the game continuously and making it un-playable while the key is held.
- For advanced students: point out that start_game() being called at the top level (outside TIC()) the first time, and from inside TIC() the second time, is a subtle but real detail

worth understanding - Lua code outside any function runs once, immediately, when the cart loads.

Level 2 - complete code:

```
-- Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
-- Checks: after a win and after a loss, R restores 0/3/5 with no leftover
enemies; R during play does nothing.
```

Level 3 - code (additions highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
-- Level 3 visual: flash the screen when the round ends (inside TIC(),
before cls(0)).
if not game_active and (time()//200)%2==0 then cls(2) else cls(0) end
```

Common Student Mistakes

- Using key(18) instead of keyp(18) - causes the restart to repeat/glitch while the key is held, exactly the bug this lesson's verification step was meant to prevent.
- Forgetting to reset the enemies table to a fresh table (enemies={}) inside start_game(), instead trying to reuse/modify the old one, leading to leftover dead entries.
- Wrapping only the spr() drawing call in "if game_active then" instead of the movement/escape logic too - enemies would stop being drawn but keep moving invisibly.